UNIVERSIDAD ADVENTISTA DE BOLIVIA

Asignatura: Tecnologías en Internet

Campus League

Plataforma web para la gestión de torneos deportivos universitarios

INTEGRANTES

Integrante	Usuario GitHub	Responsabilidad principal
Samuel Eli Ramírez Mamani	deaht911	Líder del proyecto. Estructura del monorepo, arranque de la API, modelo de partidos, rutas protegidas y gestión de equipo en el cliente, panel de administración, documentación e integración final
José Alcozer Martínez	jose-1	Autenticación y torneos. Modelo de usuarios, registro, login y OTP, CRUD de torneos y calendario, calendario público, panel del árbitro, reglas por disciplina, pruebas automáticas y diagramas
Roger Huarachi Rojas	roger69	Usuarios, equipos e interfaz. Modelo de equipos, gestión de usuarios y roles, base de OpenAPI, tema oscuro, pantallas de acceso y perfil, recuperación de contraseña, diseño responsivo y datos de demostración
Kevin Abdias Flores Escobar	Gn_Abdias	Seguridad y servicios. Modelos de torneos e inscripciones, JWT, RBAC y errores, CRUD de equipos, portal público, panel del organizador, correo, reportes PDF, cifrado AES-256 y llaves del torneo

Docente: Lic. Víctor Hugo Sarzuri Flores

Vinto, noviembre de 2026

Contenido

1. Carátula e información general 3

2. Resumen ejecutivo y alcance 4

3. Trazabilidad de requisitos 5

3.1 Requisitos funcionales (RF → código) 5
3.2 Casos de uso 6
3.3 Justificación de requisitos no funcionales 6

4. Arquitectura del sistema 8

4.1 Diagrama C4 nivel 2 (contenedores) 8
4.2 Diagrama de secuencia de autenticación y autorización 8

5. Diseño de persistencia en MongoDB 10

5.1 Colecciones y relaciones 10
5.2 Validaciones de esquema 10
5.3 Índices 10

6. Especificación de endpoints y API REST 11

6.1 Matriz de endpoints 11
6.2 Seguridad aplicada a la API 13
6.3 Reglas de negocio 13
6.4 Capturas de la aplicación 14
6.5 Diseño responsivo (RNF-005) 24

7. Configuración de entorno y variables 25

7.1 Servidor (`server/.env`) 25
7.2 Cliente (`client/.env`) 25

8. Manual de ejecución y pruebas 26

8.1 Requisitos previos 26
8.2 Instalación y ejecución local 26
8.3 Pruebas automáticas 27
8.4 Pruebas manuales realizadas (sobre el seed) 27
8.5 Flujo de trabajo Git 28

9. Conclusiones y trabajo futuro 29

9.1 Conclusiones técnicas 29
9.2 Lecciones aprendidas 29
9.3 Trabajo futuro 30

1. Carátula e información general

Campus League es una aplicación web para organizar los torneos deportivos entre las facultades de la UAB: inscripción de equipos, calendario, actas de partido, tabla de posiciones y reportes. Esta segunda entrega presenta el producto implementado como monorepo, con una SPA en React, una API REST en Express y una base de datos MongoDB. El título, los integrantes, la fecha, la asignatura y el enlace al repositorio público figuran en la carátula.

> **Repositorio público:** https://github.com/deaht911/campus-league

Tabla 1. *Ficha del proyecto*

Elemento	Detalle
Asignatura	Tecnologías en Internet
Docente	Lic. Víctor Hugo Sarzuri Flores
Fecha	Noviembre de 2026
Stack	React 19, Vite y Tailwind 4 · Node.js y Express 5 · MongoDB y Mongoose 9
Seguridad	JWT de 30 min, bcrypt (salt >= 10), RBAC de 5 roles, AES-256-GCM
Documentación de la API	Swagger UI en `/api-docs` (OpenAPI 3.0)
Ejecución	`npm run install:all`, `npm run seed`, `npm run dev` (sección 8)

Tabla 2. *Cifras del proyecto*

Indicador	Valor
Requisitos funcionales, no funcionales y casos de uso	15, 8 y 13
Endpoints documentados en Swagger	45
Colecciones de MongoDB	5
Pantallas de la aplicación	12
Código JavaScript	cerca de 8 600 líneas (servidor 4 050, cliente 4 540)
Pruebas automáticas	69, más cerca de 790 líneas de pruebas
Datos de demostración	9 torneos, 28 equipos, 188 usuarios y 78 partidos
Historial de Git	45 commits de 4 autores

Tabla 3. *Organización del repositorio*

Elemento	Ubicación
Código del servidor	`server/`
Código del cliente	`client/`
Diagramas, capturas e informe	`docs/`
Documentación interactiva de la API	`http://localhost:5000/api-docs` (Swagger UI, OpenAPI 3.0)
Manual de ejecución	`README.md` (raíz) y sección 8 de este informe

2. Resumen ejecutivo y alcance

Qué hace el sistema. Campus League cubre el ciclo completo de un torneo: el estudiante se registra con su correo institucional, el capitán crea el equipo e inscribe a sus jugadores, el organizador aprueba inscripciones y genera el calendario, el árbitro cierra el acta digital y el público consulta resultados, posiciones y goleadores. El organizador exporta los reportes en PDF y los capitanes reciben avisos por correo.

Quiénes lo usan. Cinco roles con permisos distintos (jugador, capitán, organizador, árbitro y administrador) y un portal público de solo lectura.

Qué incluye esta entrega. La aplicación completa y probada: autenticación con JWT y control de acceso por rol, persistencia en MongoDB, API de 45 endpoints documentada en Swagger, informe técnico y un conjunto de datos de demostración con 9 torneos en 5 disciplinas. La tabla de posiciones usa las reglas de cada deporte (goles, puntos o sets) y el marcador del acta se calcula con lo que el árbitro registra por jugador.

Tabla 4. *Tecnologías integradas*

Capa	Tecnología
Frontend	React 19, Vite, Tailwind CSS 4, React Router, Axios
Backend	Node.js, Express 5, express-validator, helmet, cors, express-rate-limit
Base de datos	MongoDB con Mongoose 9 (local o Atlas)
Seguridad	JWT (30 min), bcryptjs (salt >= 10), RBAC con 5 roles, OTP por correo institucional
Correo y reportes	Nodemailer (SMTP configurable) y PDFKit
Documentación	Swagger UI (OpenAPI 3.0)
Calidad	ESLint, 69 pruebas automáticas (`node:test` + Supertest)

Alcance alcanzado en esta entrega: los 15 requisitos funcionales (RF-001 a RF-015) y los 8 no funcionales de la primera entrega tienen implementación o justificación (secciones 3 y 8). Los 13 casos de uso están cubiertos.

3. Trazabilidad de requisitos

3.1 Requisitos funcionales (RF → código)

Tabla 5. *Matriz de trazabilidad de requisitos funcionales (RF → código)*

RF	Descripción	Componente frontend (React)	Endpoint / servicio backend
RF-001	Registro con correo institucional y OTP	`RegisterPage`	`POST /api/auth/register`, `verify-otp`, `resend-otp`
RF-002	Autenticación, sesión activa y recuperación de contraseña	`LoginPage`, `ForgotPasswordPage`, `UserProfile`, `AuthContext`	`POST /api/auth/login`, `GET /api/auth/me`, `POST /api/auth/forgot-password`, `reset-password`
RF-003	Asignación y modificación de roles	`AdminPanel`	`GET /api/users`, `PATCH /api/users/:id/role`, `PATCH /api/users/:id/status`
RF-004	Crear equipo (queda como capitán)	`MyTeam`	`POST /api/teams`
RF-005	Agregar, editar y dar de baja jugadores	`MyTeam`	`POST/PUT/DELETE /api/teams/:id/players`
RF-006	Elegibilidad: estudiante activo y un solo equipo por disciplina	`MyTeam`	Validación en `POST /api/teams` y `POST /api/teams/:id/players`
RF-007	Crear torneos	`OrganizerPanel`	`POST /api/tournaments`, `PUT /api/tournaments/:id`
RF-008	Inscripción solo dentro del período habilitado	`MyTeam`, `OrganizerPanel`	`POST /api/tournaments/:id/registrations`, `PATCH .../registrations/:regId`
RF-009	Calendario automático (liga o eliminación)	`OrganizerPanel`, `Calendar`	`POST /api/tournaments/:id/fixture`, `.../fixture/publish`
RF-010	Reprogramar partidos (fecha, hora, escenario)	`OrganizerPanel`	`PATCH /api/matches/:id/reschedule`, `PATCH /api/matches/:id/referee`
RF-011	Registrar el acta: el marcador es la suma automática de goles o puntos por jugador (sets en voleibol)	`RefereePanel`	`PUT /api/matches/:id/result`, `GET /api/matches/mine`
RF-012	Tabla de posiciones automática con las reglas de cada disciplina, provisional en curso y definitiva al finalizar	`Standings`	`GET /api/matches/standings/:torneoId`
RF-013	Estadísticas individuales según la disciplina (goleadores, anotadores)	`Standings`, `UserProfile`	`GET /api/matches/scorers/:torneoId`, `GET /api/matches/players/:userId/stats`
RF-014	Notificaciones por correo (inscripciones y reprogramaciones)	(transparente para el usuario)	`server/services/mailer.js`, invocado desde `reviewRegistration`, `registerTeam`, `rescheduleMatch`
RF-015	Portal público y exportación a PDF	`Home`, `Tournaments`, `Calendar`, `Standings`, botón «Exportar PDF»	`GET /api/tournaments`, `GET /api/matches`, `GET /api/reports/tournaments/:id/pdf`

3.2 Casos de uso

Tabla 6. *Casos de uso y pantallas que los implementan*

CU	Caso de uso	RF	Pantalla
CU-01	Registrarse	RF-001	Registro
CU-02	Iniciar sesión	RF-002	Login, Recuperar contraseña

CU	Caso de uso	RF	Pantalla
CU-03	Crear equipo	RF-004	Mi equipo
CU-04	Gestionar plantilla	RF-005, RF-006	Mi equipo
CU-05	Inscribir equipo en torneo	RF-008	Mi equipo
CU-06	Crear torneo	RF-007	Panel del organizador
CU-07	Generar calendario	RF-009	Panel del organizador
CU-08	Reprogramar partido	RF-010	Panel del organizador
CU-09	Registrar resultado	RF-011, RF-012	Panel del árbitro
CU-10	Consultar posiciones y estadísticas	RF-012, RF-013, RF-015	Posiciones, Calendario
CU-11	Gestionar usuarios y roles	RF-003	Panel de administración
CU-12	Exportar reporte	RF-015	Panel del organizador, Posiciones
CU-13	Enviar notificación	RF-014	Servicio de correo

3.3 Justificación de requisitos no funcionales

Tabla 7. *Justificación de cumplimiento de los requisitos no funcionales*

RNF	Requisito	Cómo se cumple	Evidencia
RNF-001	API < 200 ms; páginas < 2 s	Índices en las consultas frecuentes (sección 5), paginación en listados, cálculo de posiciones con una consulta, build de Vite (JS ~124 kB gzip)	Los logs de desarrollo muestran 3–7 ms en listados de torneos, 9–35 ms en partidos y posiciones y ~100–180 ms en login (el costo es el hash bcrypt, deliberado)
RNF-002	JWT de 30 min, bcrypt y cifrado AES-256 de datos sensibles	JWT con `JWT_EXPIRE=30m`; contraseñas con bcryptjs (salt >= 10); OTP guardado como hash; el código estudiantil se cifra en la aplicación con **AES-256-GCM** (clave `ENCRYPTION_KEY` de 256 bits derivada con HKDF, IV derivado con HMAC-SHA256 para conservar el índice único y la búsqueda exacta, etiqueta de autenticación que detecta alteraciones). MongoDB Atlas añade cifrado del almacenamiento en reposo	`server/services/crypto.js`, `server/models/User.js`, pruebas `crypto.test.js`
RNF-003	Comunicación sobre HTTPS (TLS 1.3)	La app no termina TLS: se despliega detrás de un proxy o plataforma con HTTPS. helmet aplica cabeceras seguras y CORS se limita a `FRONTEND_URL`	Configuración en `server/server.js`
RNF-004	WCAG 2.1 AA	Tema oscuro con texto claro sobre fondos oscuros (contraste alto), etiquetas `label`, atributos `aria-*`, roles `alert/status`, foco visible, navegación por teclado y mensajes de error claros	Capturas de la sección 6
RNF-005	Responsive 375–1440 px	Diseño con utilidades responsive de Tailwind, menú hamburguesa, tablas con scroll horizontal	Navbar y paneles
RNF-006	Disponibilidad 99,5 % y respaldos diarios	Depende del despliegue: `GET /api/health` permite monitoreo y MongoDB Atlas ofrece respaldos automáticos	Endpoint de salud
RNF-007	500 usuarios concurrentes	API sin estado (JWT, sin sesiones en memoria), consultas indexadas, paginación y límite de intentos en autenticación	Arquitectura de la sección 4

RNF	Requisito	Cómo se cumple	Evidencia
RNF-008	Arquitectura modular y OpenAPI 3.0	Capas `models`, `controllers`, `routes`, `middlewares`, `services`; Swagger UI con los 45 endpoints	`server/swagger/swagger.yaml`

4. Arquitectura del sistema

Monorepo con cliente (SPA), API REST y base de datos documental. Los diagramas fuente están en `docs/diagrams/`.

4.1 Diagrama C4 nivel 2 (contenedores)

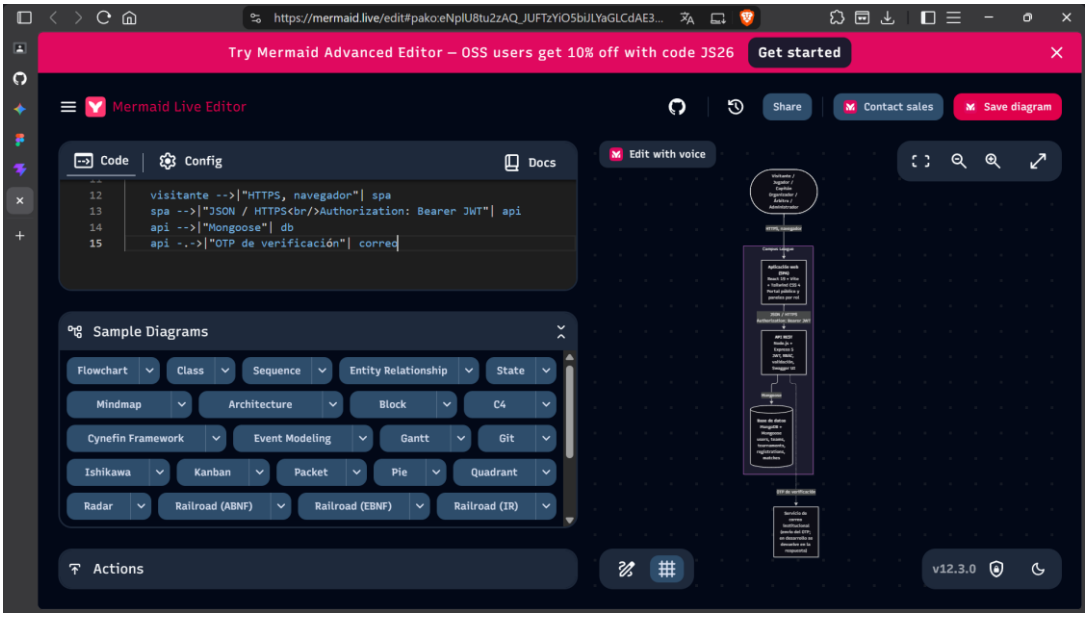


Figura 1. *Diagrama C4 nivel 2*

La SPA (React) se comunica con la API REST (Express) por JSON/HTTPS enviando `Authorization: Bearer <JWT>`. La API persiste en MongoDB mediante Mongoose y envía correos (OTP y notificaciones) a través de un servidor SMTP configurable. Código fuente: `diagrams/c4-nivel2-contenedores.md`.

4.2 Diagrama de secuencia de autenticación y autorización

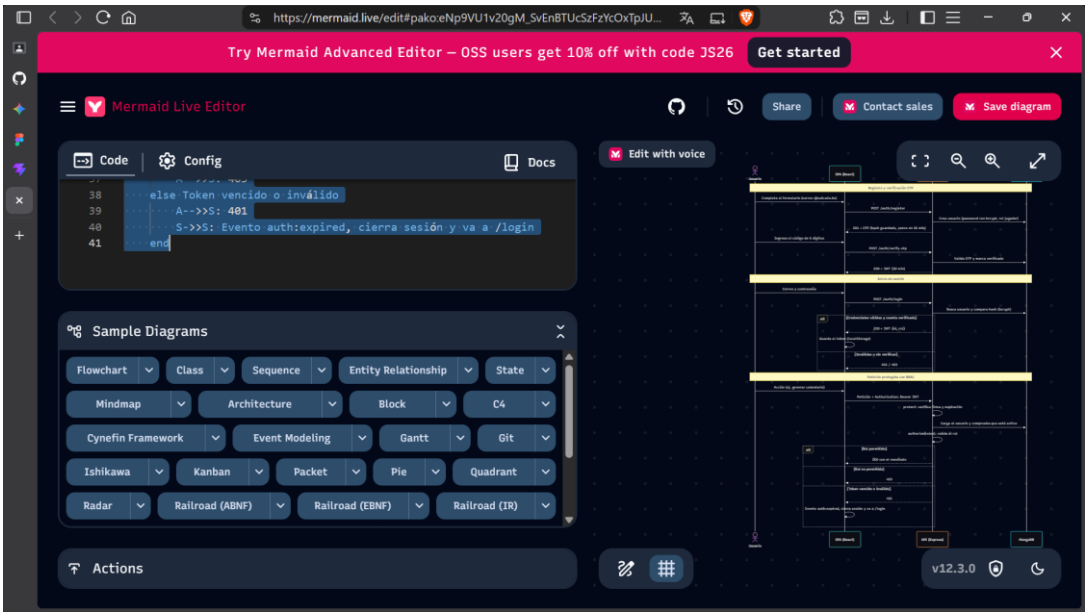


Figura 2. *Secuencia JWT*

El flujo cubre el registro con OTP, el login (verificación bcrypt y firma del JWT con `id` y `rol`), el almacenamiento del token en el cliente (`localStorage`), el envío en el encabezado `Authorization: Bearer` y las validaciones de

`protect` (firma, expiración, cuenta activa) y `authorize(roles)`. Un 401 hace que el cliente borre el token y vuelva a `/login`. Código fuente: `diagrams/secuencia-jwt.md`.

5. Diseño de persistencia en MongoDB

Base de datos `campus_league` con cinco colecciones y Mongoose como ODM. El diagrama entidad-relación está en `diagrams/modelo-datos.md`.

5.1 Colecciones y relaciones

Tabla 8. *Colecciones de MongoDB y sus relaciones*

Colección	Contenido	Relaciones (referencias ObjectId)
`users`	Cuenta, rol, facultad, código estudiantil, OTP hasheado	—
`teams`	Equipo y plantilla (`jugadores[]` embebido)	`capitan` → users; `jugadores[].usuario` → users
`tournaments`	Torneo, fechas, cupos, reglamento, estado	`organizador` → users
`registrations`	Inscripción de un equipo a un torneo	`torneo`, `equipo`, `inscritoPor`, `revisadoPor`
`matches`	Partido, marcador, acta (`estadisticas[]` embebido), historial de reprogramación	`torneo`, `local`, `visitante`, `arbitro`

Se embebieron la plantilla, las estadísticas por partido y el historial porque se leen siempre junto al documento padre; se usaron referencias donde la entidad tiene vida propia (usuarios, equipos, torneos).

5.2 Validaciones de esquema

Tabla 9. *Validaciones de esquema de Mongoose por modelo*

Modelo	Validaciones
`User`	Correo con regex `@uab.edu.bo` (único), contraseña mínima de 8 caracteres y `select: false`, rol en enum de 5 valores, código estudiantil con regex, hash bcrypt con salt >= 10 en `pre('save')`, código estudiantil cifrado con AES-256-GCM (setter/getter transparente), `toJSON` sin `password` ni `otp`
`Team`	Nombre 3–60, disciplina en enum, capitán obligatorio, sin jugadores repetidos, sin camisetas repetidas entre activos, máximo 30 jugadores
`Tournament`	Fecha de fin >= inicio, fin de inscripción >= inicio, 4–64 equipos, formato y estado en enum, reglamento obligatorio
`Registration`	`aceptaReglamento` debe ser `true`, estado en enum
`Match`	Local distinto de visitante, marcador >= 0, estado y fase en enum, goles individuales coherentes con el marcador

5.3 Índices

Tabla 10. *Índices definidos para rendimiento*

Colección	Índices
`users`	`correo` único, `codigoEstudiantil` único y disperso, `rol`, `facultad + rol`, `createdAt`
`teams`	`nombre + disciplina` único, `capitan`, `disciplina + activo`, `jugadores.usuario + disciplina`
`tournaments`	`nombre + disciplina` único, `disciplina + estado`, `organizador`, `fechaInicio`
`registrations`	`torneo + equipo` único, `torneo + estado`, `equipo`, `inscritoPor`
`matches`	`torneo + fecha + escenario` único, `torneo + fecha`, `torneo + estado`, `local`, `visitante`, `arbitro + estado`

6. Especificación de endpoints y API REST

La documentación interactiva está en **Swagger UI**: `http://localhost:5000/api-docs` (JSON en `/api-docs.json`; fuente en `server/swagger/swagger.yaml`). Todos los endpoints cuelgan de `/api`. Las respuestas de error usan el formato `{ "success": false, "message": "...", "errors": [...] }`.

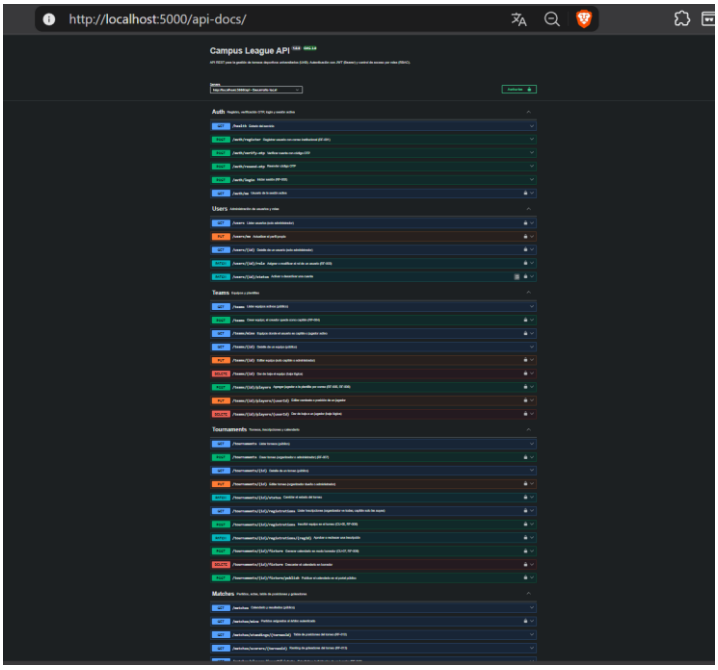


Figura 3. *Swagger UI*

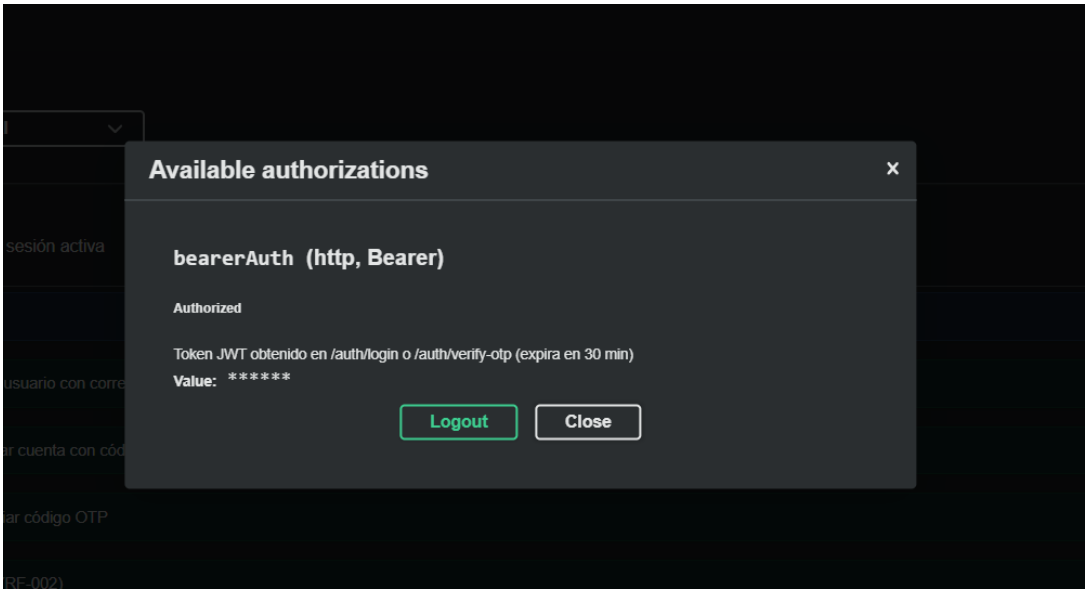


Figura 4. *Swagger: autorización con JWT*

6.1 Matriz de endpoints

> **Códigos de uso común.** 200 OK, 201 creado, 400 datos inválidos, 401 sin sesión o token inválido, 403 rol sin permiso, 404 no encontrado, 409 conflicto (duplicado o choque de horario), 429 demasiados intentos. Un «?» junto a un parámetro indica que es opcional.

Tabla 11. *Matriz de endpoints de la API REST*

Método	Ruta	Parámetros	Códigos HTTP	Acceso (RBAC)
POST	`/api/auth/register`	cuerpo: `nombre`, `correo`, `password`	201, 400, 409	Público
POST	`/api/auth/verify-otp`	cuerpo: `correo`, `codigo`	200, 400	Público
POST	`/api/auth/resend-otp`	cuerpo: `correo`	200, 400	Público
POST	`/api/auth/login`	cuerpo: `correo`, `password`	200, 400, 401, 403	Público
GET	`/api/health`	–	200	Público
GET	`/api/auth/me`	–	200, 401	Sesión
POST	`/api/auth/forgot-password`	cuerpo: `correo`	200, 400, 429	Público
POST	`/api/auth/reset-password`	cuerpo: `correo`, `codigo`, `password`	200, 400, 429	Público
GET	`/api/teams`	`disciplina`?; `facultad`?; `q`?	200	Público
POST	`/api/teams`	cuerpo: `nombre`, `disciplina`	201, 400, 401, 403, 409	Jugador, Capitán, Admin
GET	`/api/teams/mine`	–	200, 401	Sesión
GET	`/api/teams/{id}`	`id`	200, 400, 404	Público
PUT	`/api/teams/{id}`	`id`	200, 401, 403, 404	Capitán del equipo, Admin
DELETE	`/api/teams/{id}`	`id`	200, 401, 403, 404	Capitán del equipo, Admin
POST	`/api/teams/{id}/players`	`id`; cuerpo: `correo`	201, 400, 401, 403, 404, 409	Capitán del equipo, Admin
PUT	`/api/teams/{id}/players/{userId}`	`id`; `userId`	200, 401, 403, 404	Capitán del equipo, Admin
DELETE	`/api/teams/{id}/players/{userId}`	`id`; `userId`	200, 400, 401, 403, 404	Capitán del equipo, Admin
GET	`/api/tournaments`	`disciplina`?; `estado`?	200	Público
POST	`/api/tournaments`	cuerpo: `nombre`, `disciplina`, `formato`, `fechaInicio`, `fechaFin`, `inscripcionInicio`, `inscripcionFin`, `maxEquipos`, `reglamento`	201, 400, 401, 403	Organizador, Admin
GET	`/api/tournaments/{id}`	`id`	200, 404	Público
PUT	`/api/tournaments/{id}`	`id`	200, 400, 401, 403, 404	Organizador dueño, Admin
PATCH	`/api/tournaments/{id}/status`	`id`; cuerpo: `estado`	200, 401, 403, 404	Organizador dueño, Admin
GET	`/api/tournaments/{id}/registrations`	`id`; `estado`?	200, 401, 403	Organizador dueño, Admin; Capitán (las suyas)
POST	`/api/tournaments/{id}/registrations`	`id`; cuerpo: `equipo`, `aceptaReglamento`	201, 400, 401, 403, 404, 409	Capitán, Admin
PATCH	`/api/tournaments/{id}/registrations/{regId}`	`id`; `regId`; cuerpo: `estado`	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
POST	`/api/tournaments/{id}/fixture`	`id`	201, 400, 401, 403, 409	Organizador dueño, Admin
DELETE	`/api/tournaments/{id}/fixture`	`id`	200, 400, 401, 403	Organizador dueño, Admin

Método	Ruta	Parámetros	Códigos HTTP	Acceso (RBAC)
POST	`/api/tournaments/{id}/fixture/publish`	`id`	200, 400, 401, 403	Organizador dueño, Admin
GET	`/api/matches`	`torneo?`; `equipo?`; `estado?`; `fase?`; `jornada?`; `desde?`; `hasta?`; `limit?`	200, 400	Público (borrador solo organizador)
GET	`/api/matches/mine`	`estado?`	200, 401, 403	Árbitro, Admin
GET	`/api/matches/standings/{torneoId}`	`torneoId`; `torneoId?`	200, 400, 404	Público
GET	`/api/matches/scorers/{torneoId}`	`torneoId`; `torneoId?`; `limit?`	200, 400	Público
GET	`/api/matches/players/{userId}/stats`	`userId`; `torneo?`	200, 400	Público
GET	`/api/matches/{id}`	`id`	200, 404	Público (borrador solo organizador)
PATCH	`/api/matches/{id}/referee`	`id`; cuerpo: `arbitro`	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
PATCH	`/api/matches/{id}/reschedule`	`id`	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
PATCH	`/api/matches/{id}/reopen`	`id`	200, 400, 401, 403, 404	Organizador dueño, Admin
PUT	`/api/matches/{id}/result`	`id`	200, 400, 401, 403, 404, 409	Árbitro asignado, Admin
GET	`/api/users`	`rol?`; `activo?`; `q?`; `page?`; `limit?`	200, 401, 403	Admin
PUT	`/api/users/me`	–	200, 400, 401	Sesión
GET	`/api/users/{id}`	`id`	200, 401, 403, 404	Admin
PATCH	`/api/users/{id}/role`	`id`; cuerpo: `rol`	200, 400, 401, 403, 404	Admin
PATCH	`/api/users/{id}/status`	`id`; cuerpo: `activo`	200, 400, 401, 403, 404	Admin
GET	`/api/users/referees`	–	200, 401, 403	Organizador, Admin
GET	`/api/reports/tournaments/{id}/pdf`	`id`; `tipo?`	200, 400, 401, 403, 404	Organizador dueño, Admin

6.2 Seguridad aplicada a la API

- `protect` verifica firma y expiración del JWT y que la cuenta siga activa; `authorize(...roles)` aplica el RBAC.
- El rol nunca se acepta desde el cliente: todo registro inicia como `jugador`.
- Datos personales sensibles (código estudiantil) cifrados con AES-256-GCM antes de guardarse; la API los devuelve descifrados solo a quien tiene acceso.
- Validación de entrada con express-validator en todas las rutas, `helmet`, CORS restringido y límite de intentos en `/api/auth`.
- Respuestas genéricas en `resend-otp` y `forgot-password` para no revelar si un correo existe.

6.3 Reglas de negocio

- Plantilla mínima y jugadores habilitados para inscribirse; lista de espera al llenarse los cupos.

- Calendario con mínimo de 4 equipos aprobados, liga (algoritmo del círculo, ida y vuelta opcional) o eliminación directa, y detección de choques de escenario, equipos y árbitro con una ventana de 90 minutos.
- **Marcador automático:** en fútbol, futsal y baloncesto el marcador es la suma de goles o puntos registrados por jugador; el servidor rechaza un marcador enviado que no coincida. En voleibol se registran los sets (el ganador tiene 3 y el perdedor 0, 1 o 2). En baloncesto y en eliminación directa no se admiten empates. Walkover 3-0; reapertura dentro de 24 horas.
- El torneo pasa a *finalizado* al cerrarse su último partido (liga) o su final (eliminación directa).
- Reprogramar no admite fechas pasadas y conserva historial.
- **Posiciones por disciplina**, calculadas en cada consulta con los partidos cerrados:

Tabla 12. *Reglas de puntuación y estadísticas por disciplina*

Disciplina	Puntos de tabla	Columnas	Estadística individual
Fútbol y futsal	Victoria 3, empate 1, derrota 0	PJ PG PE PP GF GC DG PTS	Goleadores
Baloncesto	Victoria 2, derrota 1 (sin empates)	PJ PG PP PF PC DP PTS	Anotadores (puntos)
Voleibol	3-0 y 3-1: 3 pts; 3-2: 2 pts al ganador y 1 al perdedor; derrota 0-3 o 1-3: 0	PJ PG PP SG SC DS PTS	No aplica (se registran sets)
Eliminación directa	—	Llaves por fase: cuartos, semifinales, tercer puesto y final	Según la disciplina

- **Estado del torneo en la pantalla de posiciones:** *no iniciado* (fecha de inicio prevista), *en curso* (posiciones provisionales, avance de partidos jugados, líder y actualización automática cada 30 s) y *finalizado* (resultados definitivos y campeón).

6.4 Capturas de la aplicación

Portal público

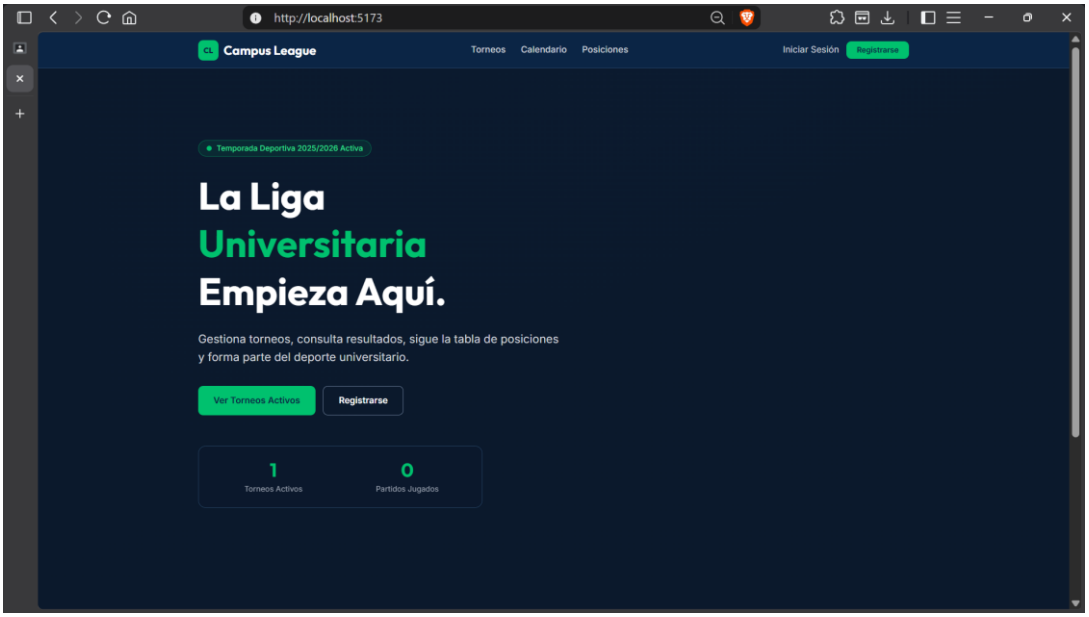


Figura 5. *Inicio*

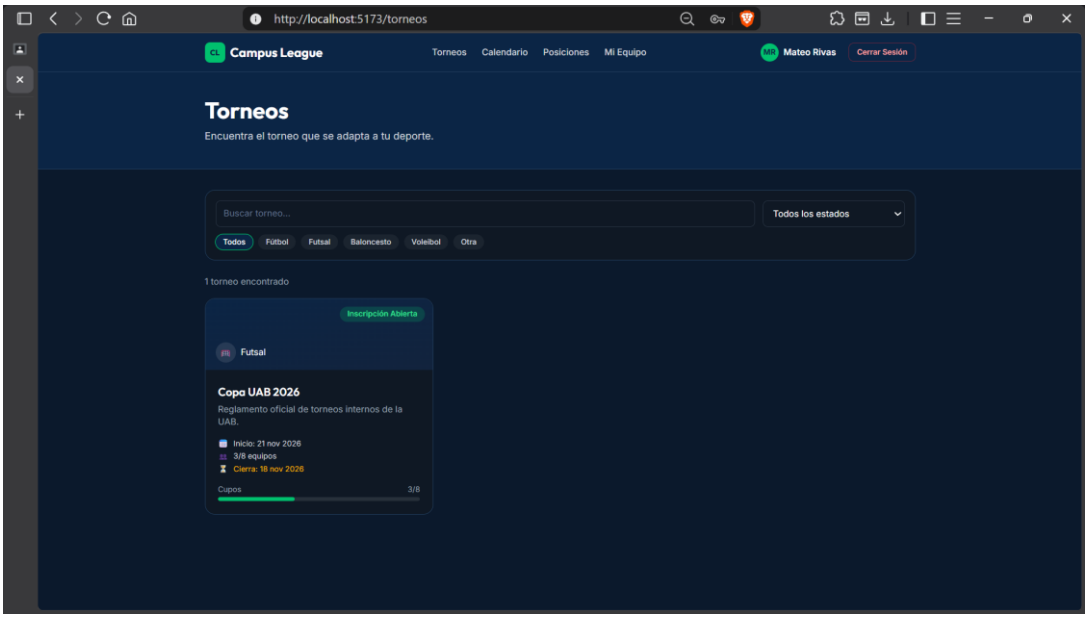


Figura 6. *Catálogo de torneos*

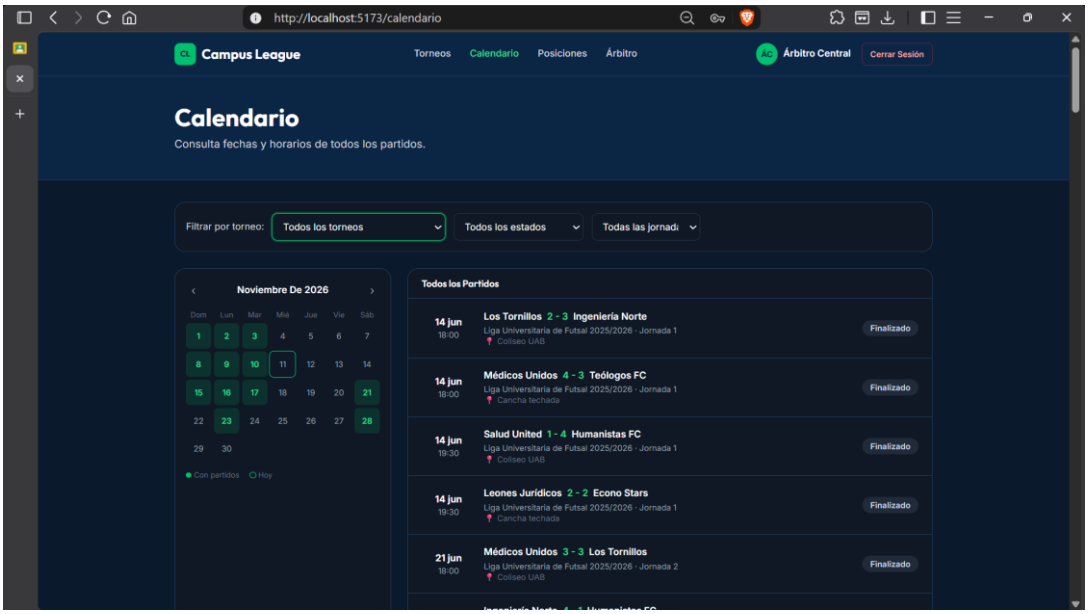


Figura 7. *Calendario con partidos de varios torneos*

Posiciones y estado del torneo por disciplina (RF-012, RF-013)

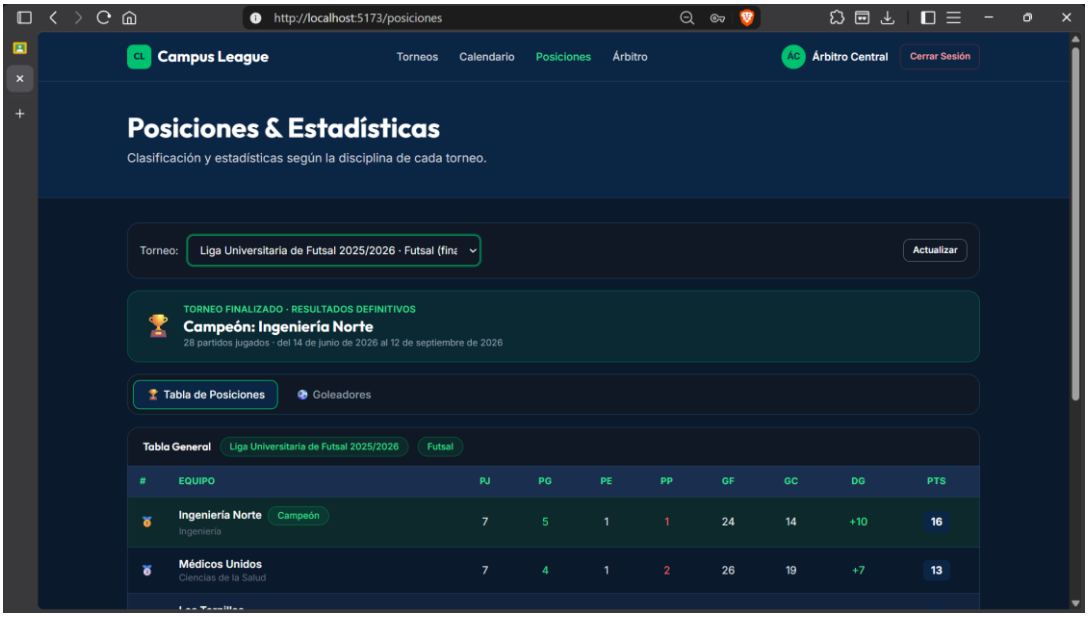


Figura 8. *Torneo finalizado de futsal: campeón y tabla final*

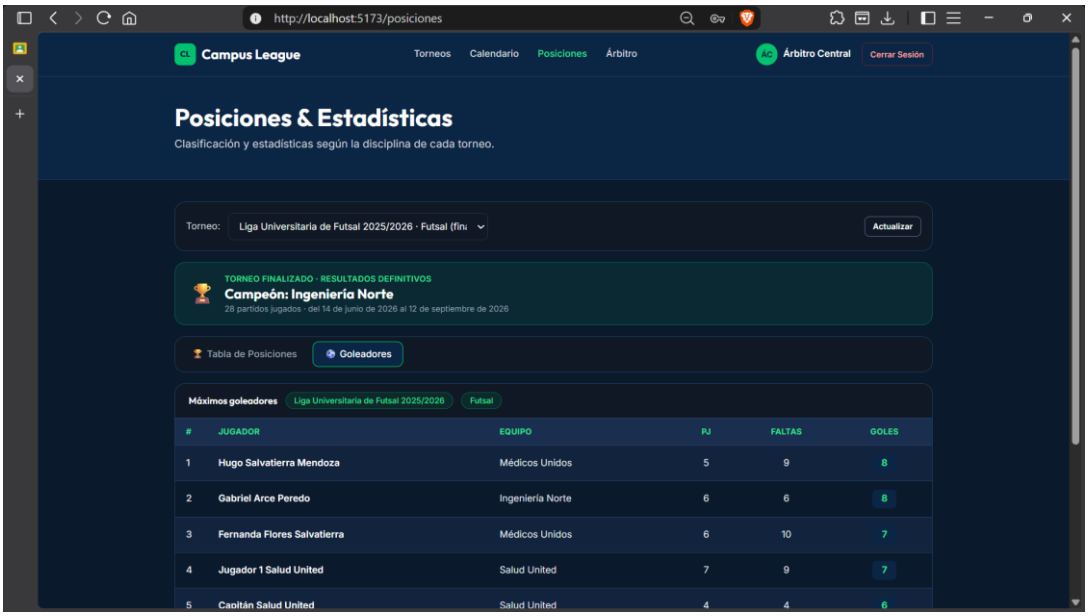


Figura 9. *Goleadores del torneo de futsal*

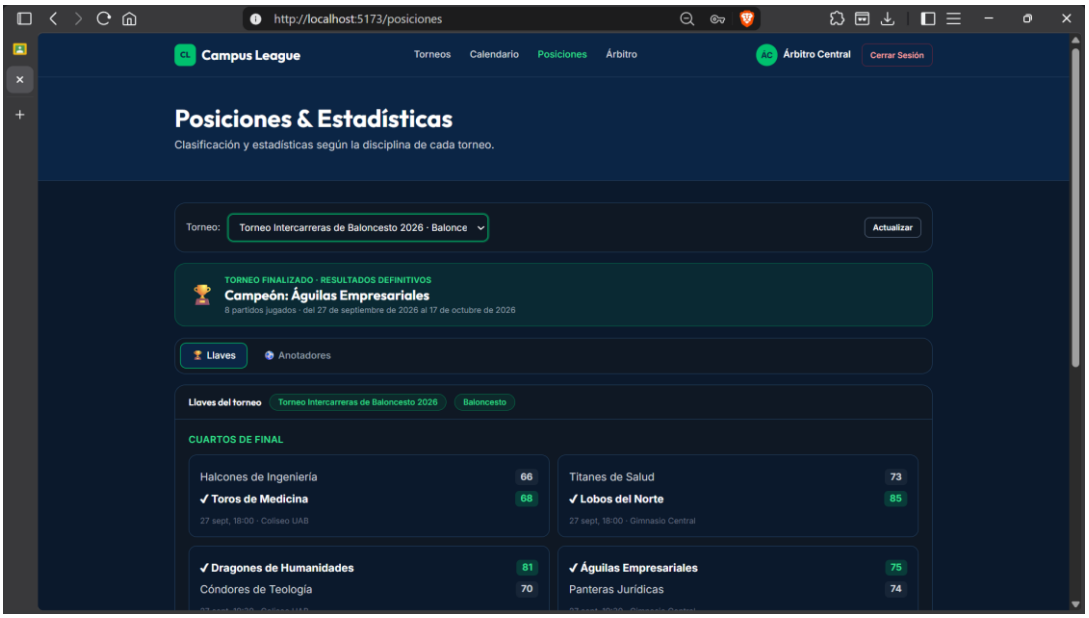


Figura 10. *Torneo de baloncesto en eliminación directa: llaves y campeón*



Figura 11. *Voleibol en curso: posiciones provisionales con sets*

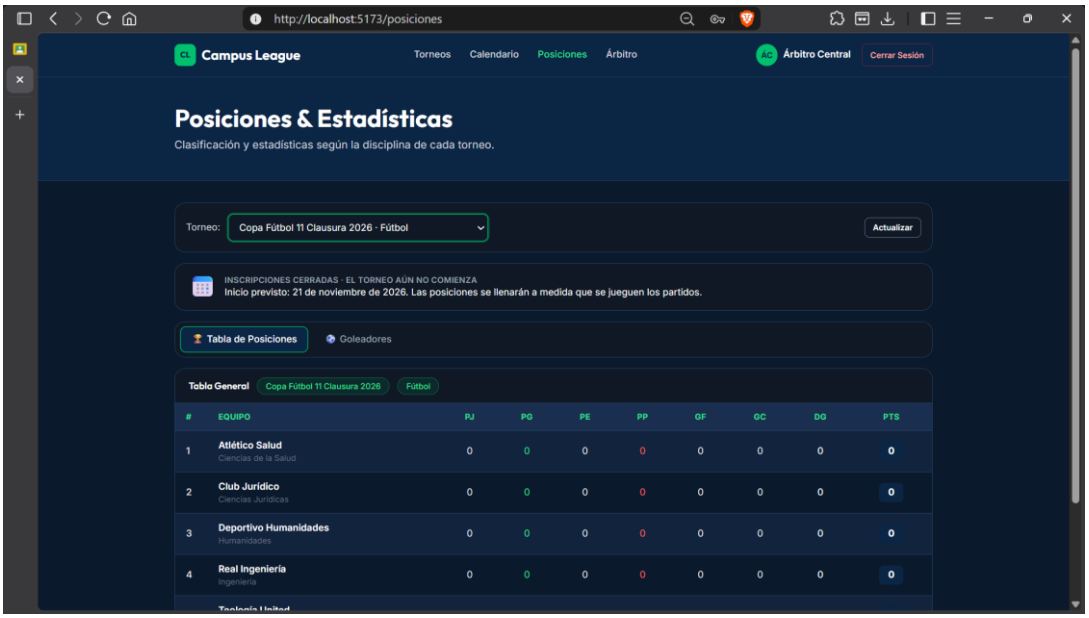


Figura 12. *Torneo que aún no comienza*

Autenticación y cuenta (RF-001, RF-002)

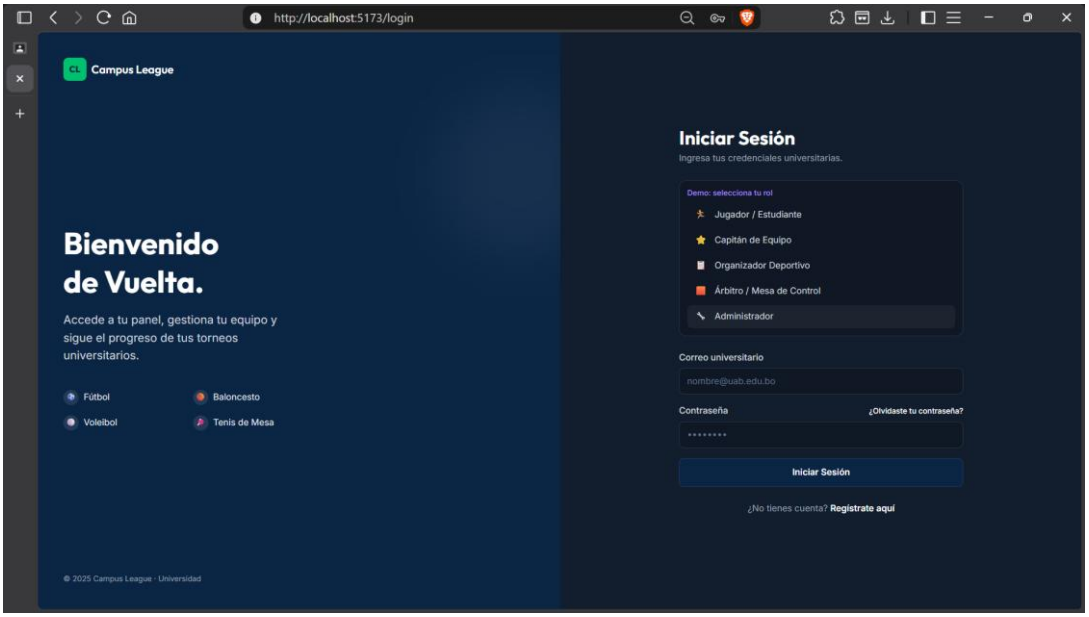


Figura 13. *Inicio de sesión*

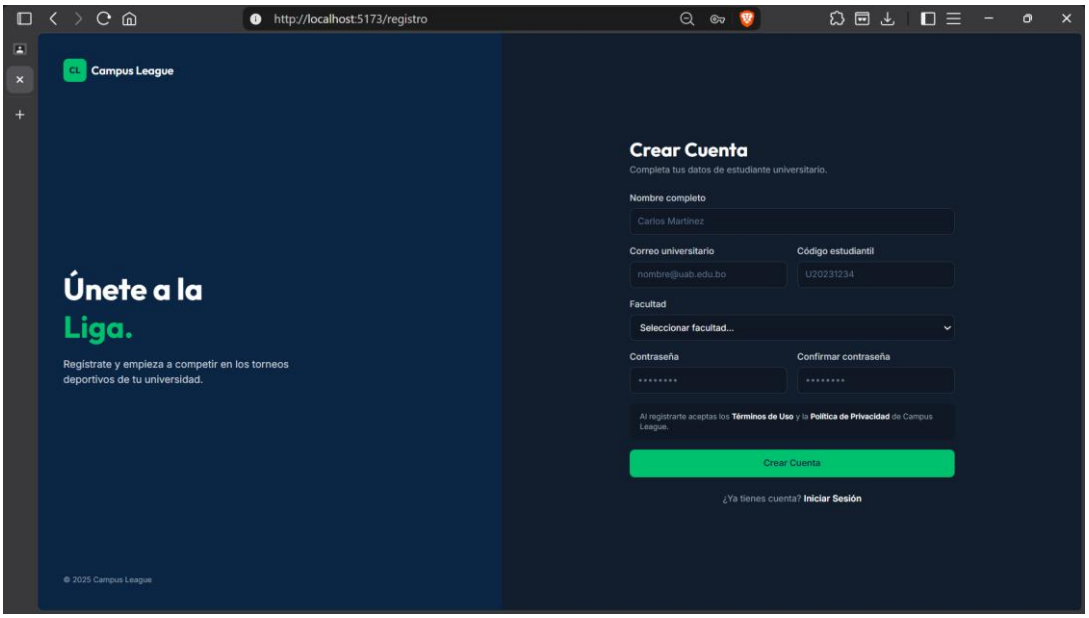


Figura 14. *Registro con correo institucional*

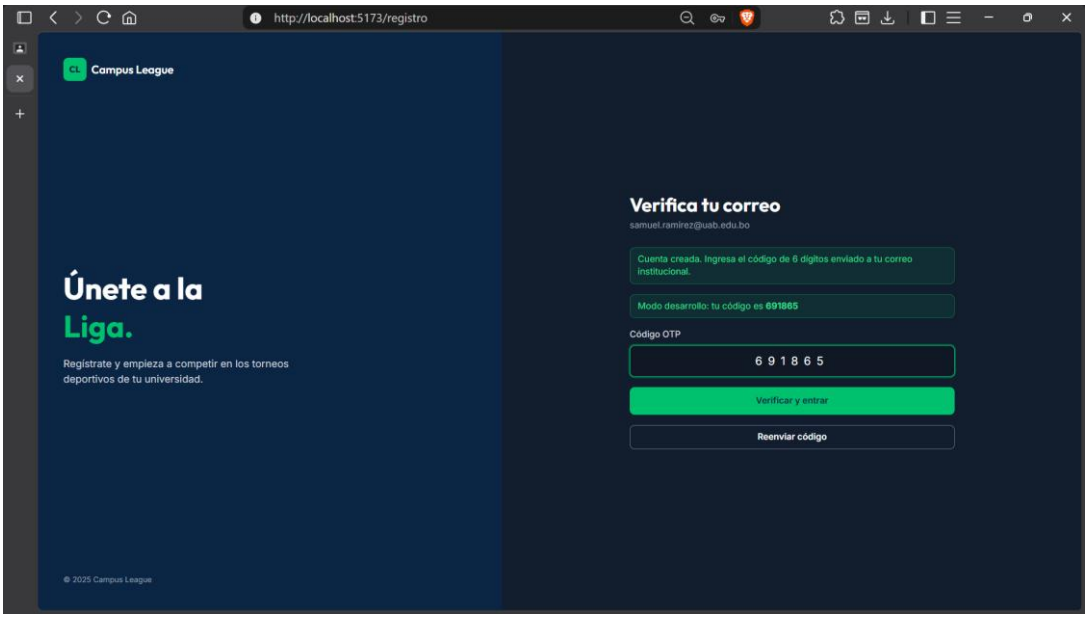


Figura 15. *Verificación con código OTP*

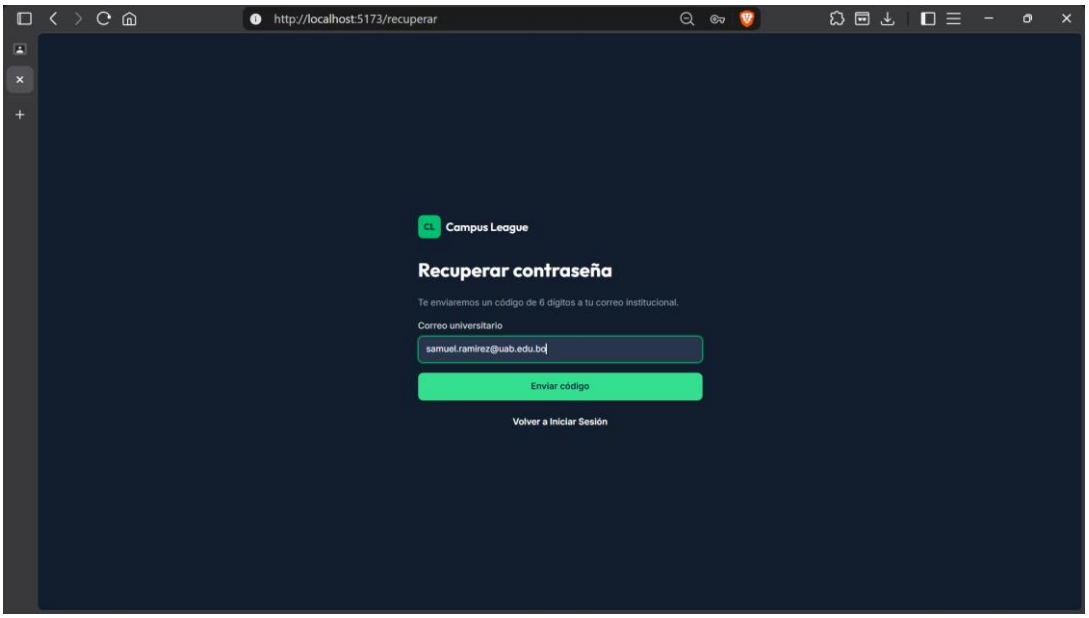


Figura 16. *Recuperación de contraseña: solicitud del código*

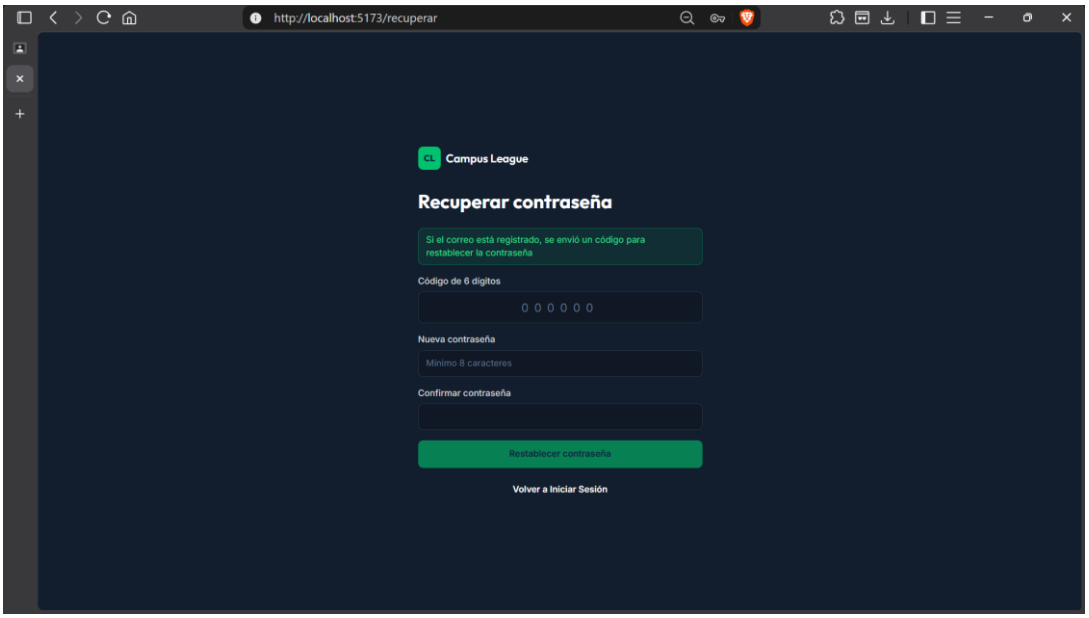


Figura 17. *Recuperación de contraseña: nuevo código y contraseña*

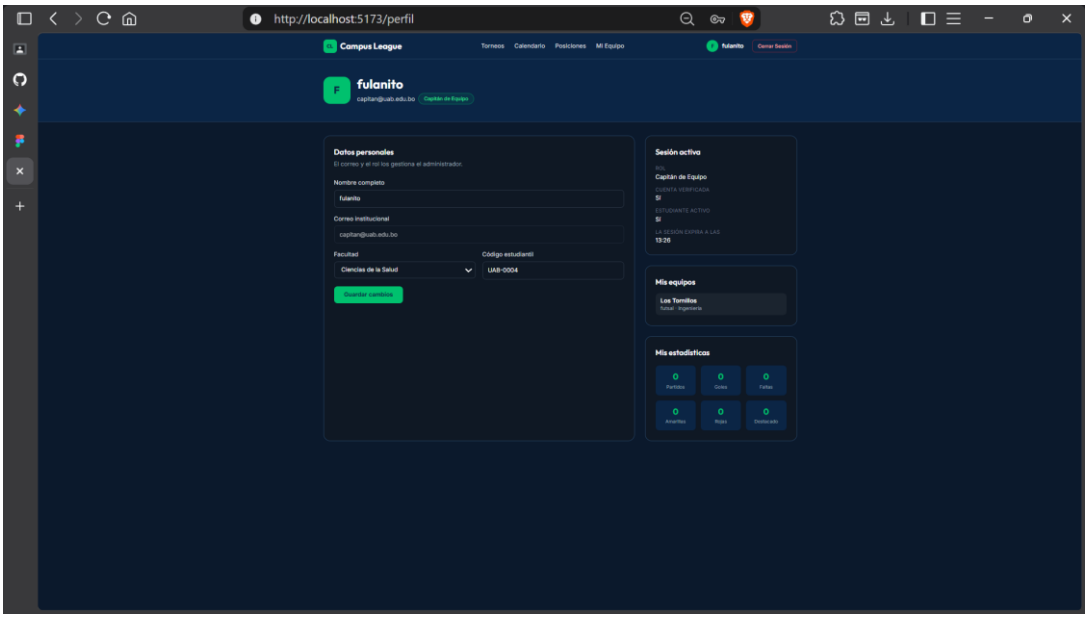


Figura 18. *Perfil de usuario*

Gestión por rol

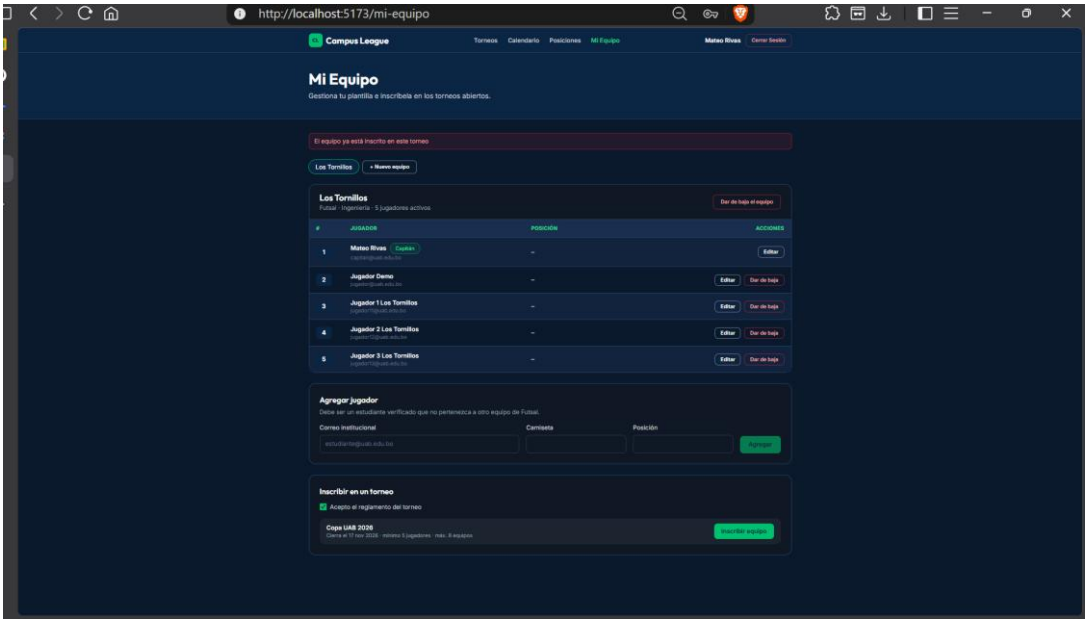


Figura 19. *Gestión de equipo (capitán)*

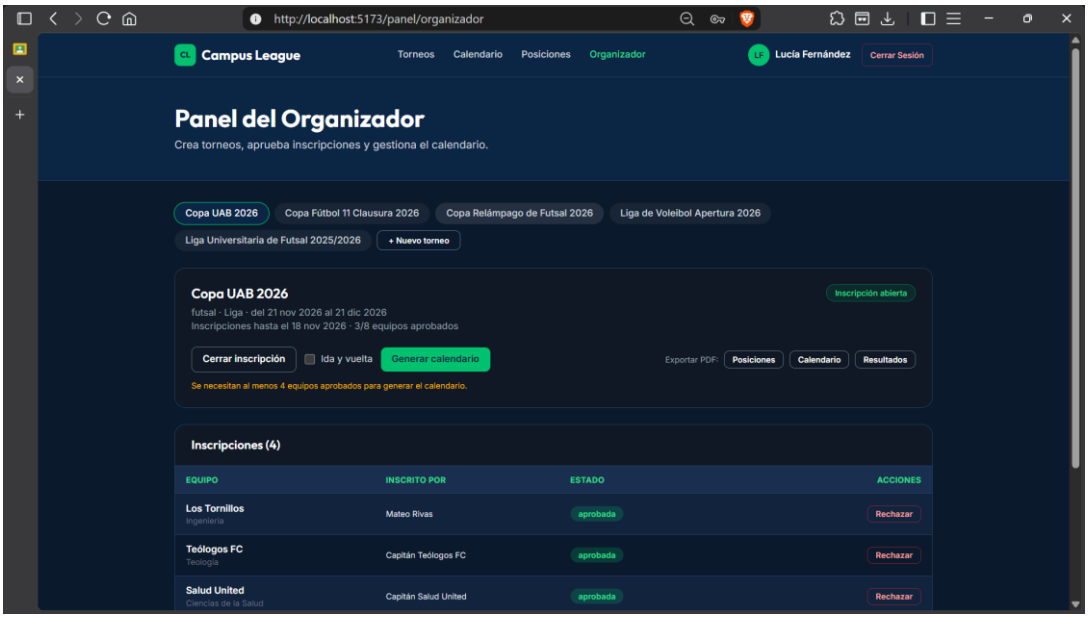


Figura 20. *Panel del organizador (varios torneos a su cargo)*

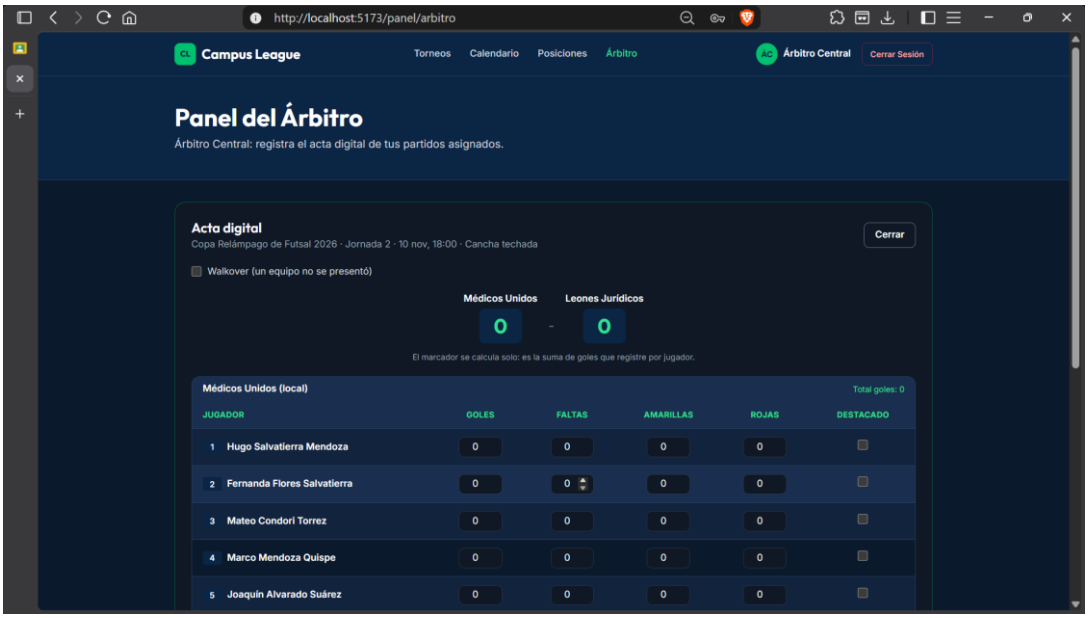


Figura 21. *Acta digital de futsal: el marcador se calcula solo con los goles de cada jugador*

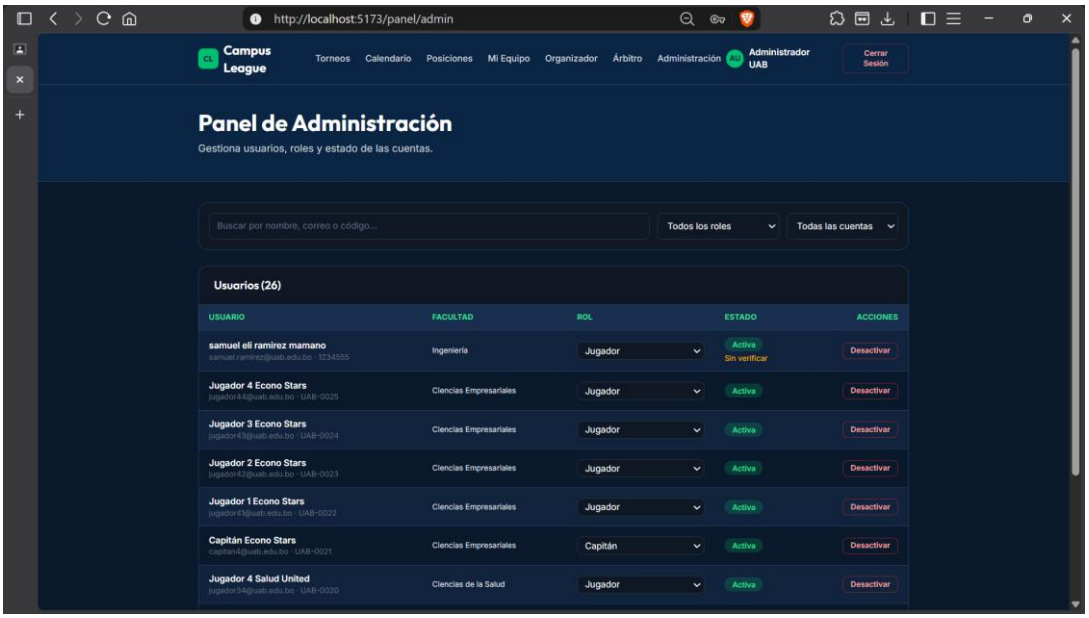


Figura 22. *Panel de administración*

Reportes y seguridad (RF-015, RNF-002)

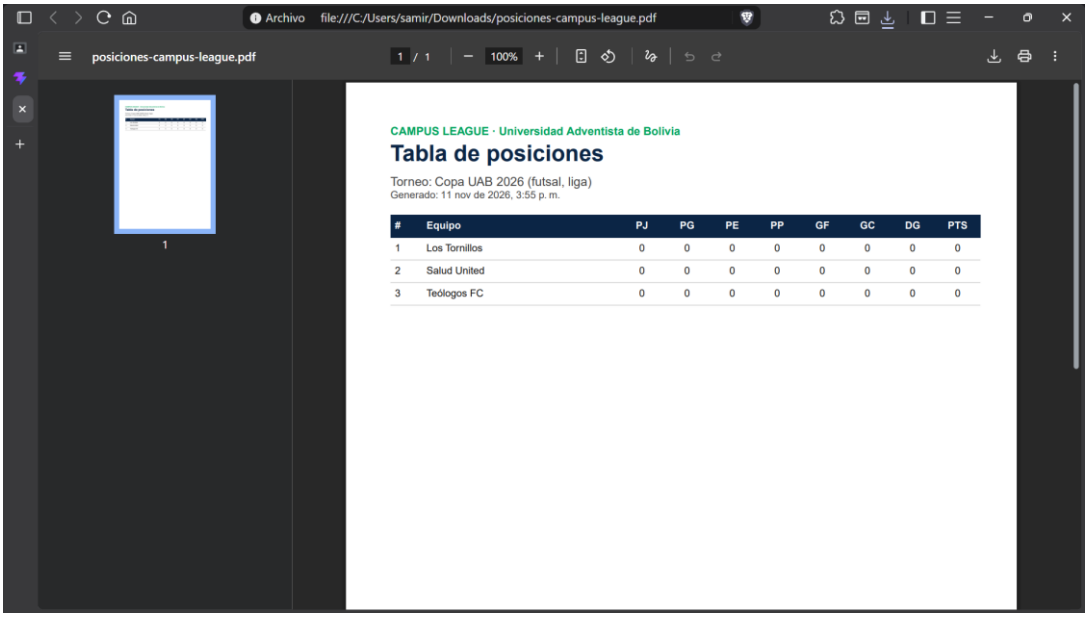


Figura 23. *Reporte de posiciones exportado en PDF*



Figura 24. *Código estudiantil cifrado con AES-256-GCM en MongoDB*

6.5 Diseño responsivo (RNF-005)

La interfaz se adapta de 375 a 1440 px: en pantallas menores de 1024 px la navegación se pliega en un menú desplegable, los filtros y formularios se apilan en una columna, las tarjetas ocupan todo el ancho y las tablas se desplazan horizontalmente dentro de su contenedor sin mover la página. Debajo se muestran capturas a 375 px: menú desplegado e inicio, calendario, tabla de posiciones y gestión de equipo.

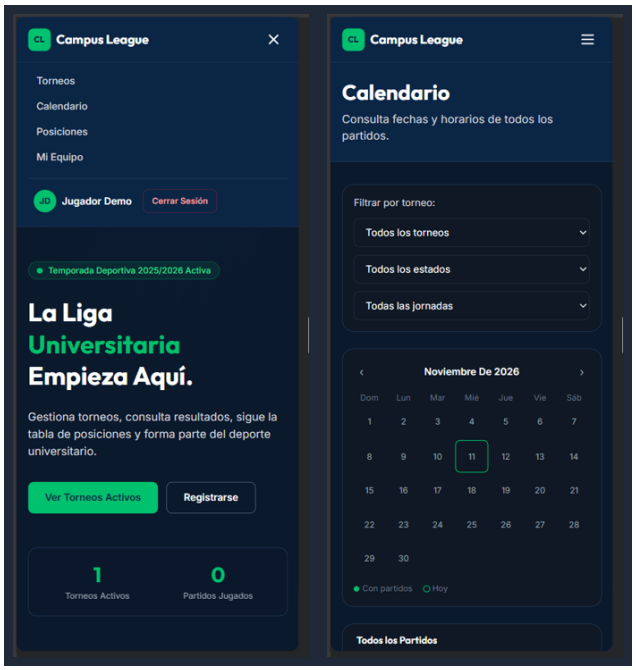


Figura 25. *Menú, inicio y calendario a 375 px*

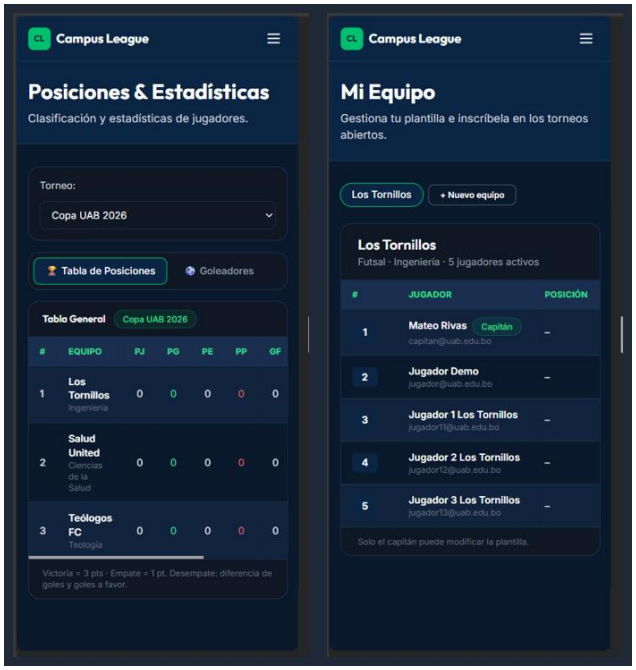


Figura 26. *Posiciones y gestión de equipo a 375 px*

7. Configuración de entorno y variables

Los archivos `.env.example` (raíz, `client/` y `server/`) documentan las variables sin credenciales reales. Cada integrante copia `server/.env.example` a `server/.env` y `client/.env.example` a `client/.env`. `.env` está excluido por `.gitignore`.

7.1 Servidor (`server/.env`)

Tabla 13. *Variables de entorno del servidor*

Variable	Descripción	Local	Producción
`PORT`	Puerto de la API	`5000`	Lo asigna la plataforma
`NODE_ENV`	Entorno. En `production` el OTP no se devuelve en las respuestas	`development`	`production`
`MONGO_URI`	Cadena de conexión	`mongodb://127.0.0.1:27017/campus_league`	URI de MongoDB Atlas (`mongodb+srv://...`)
`JWT_SECRET`	Clave de firma de tokens	Cadena larga y aleatoria	Secreto único generado con `crypto.randomBytes(64)`
`JWT_EXPIRE`	Vigencia del token	`30m`	`30m`
`BCRYPT_SALT_ROUNDS`	Costo de bcrypt (mínimo 10)	`10`	`10` o más
`ENCRYPTION_KEY`	Clave AES-256 (64 caracteres hexadecimales) para cifrar datos personales	Vacía (se deriva de `JWT_SECRET`)	Obligatoria: `openssl rand -hex 32`
`FRONTEND_URL`	Orígenes CORS permitidos (separados por coma)	`http://localhost:5173`	URL pública del cliente
`API_PROD_URL`	URL pública de la API para Swagger	—	`https://api.midominio.com`
`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`	Servidor de correo. Sin `SMTP_HOST` los correos se simulan en consola	vacío	Servidor SMTP real
`MAIL_FROM`	Remitente de los correos	`Campus League <no-reply@uab.edu.bo>`	Correo institucional
`RATE_LIMIT_MAX`	Intentos por IP cada 15 min en `/api/auth`	`100`	Valor más estricto (p. ej. `20`)

7.2 Cliente (`client/.env`)

Tabla 14. *Variables de entorno del cliente*

Variable	Descripción	Local	Producción
`VITE_API_URL`	URL base de la API	`http://localhost:5000/api`	`https://api.midominio.com/api`

8. Manual de ejecución y pruebas

8.1 Requisitos previos

Node.js 20 o superior, npm, Git y MongoDB local (`mongod`) o una cuenta de MongoDB Atlas.

8.2 Instalación y ejecución local

```
git clone https://github.com/deaht911/campus-league.git
cd campus-league
npm run install:all
cp server/.env.example server/.env
cp client/.env.example client/.env
# editar server/.env: MONGO_URI y JWT_SECRET
npm run seed     # carga datos de demostración (borra la base)
npm run dev      # API en :5000 y cliente en :5173
```

Tabla 15. *Direcciones locales de la aplicación*

URL	Contenido
`http://localhost:5173`	Aplicación
`http://localhost:5000/api-docs`	Swagger UI
`http://localhost:5000/api/health`	Estado del servicio

Cuentas de demostración (contraseña `Campus2026!`): `admin@`, `organizador@`, `organizador2@`, `arbitro@`, `capitan@` y `jugador@uab.edu.bo`. Los capitanes de los demás equipos (`capitan2@` a `capitan28@`) se listan al terminar `npm run seed`.

> **Correo simulado.** Sin `SMTP_HOST` en `server/.env`, el sistema no envía correos: imprime el mensaje en la consola del servidor y, fuera de producción, muestra los códigos OTP y de recuperación en pantalla. Con un servidor SMTP configurado envía los correos reales y deja de mostrar los códigos.

Datos de demostración

El seed carga 9 torneos en 5 disciplinas, 28 equipos, 188 usuarios y 78 partidos (resultados simulados pero deterministas) para poder recorrer el historial completo:

Tabla 16. *Torneos cargados por el seed de demostración*

Torneo	Disciplina y formato	Estado	Qué permite ver
Liga Universitaria de Futsal 2025/2026	Futsal, liga de 8 equipos	Finalizado	Tabla final, campeón, goleadores y 28 resultados
Torneo Intercarreras de Baloncesto 2026	Baloncesto, eliminación directa	Finalizado	Cuartos, semifinales, tercer puesto y final, con campeón
Liga de Voleibol Apertura 2026	Voleibol, liga de 6 equipos	En curso	Resultados parciales, actas pendientes y partidos futuros
Liga Express de Baloncesto 2026	Baloncesto, liga de 4 equipos	En curso	Actas pendientes: el marcador se suma con los puntos por jugador
Copa Relámpago de Futsal 2026	Futsal, liga de 4 equipos	En curso	Actas pendientes: el marcador se suma con los goles por jugador
Copa Fútbol 11 Clausura 2026	Fútbol, liga de 6 equipos	Inscripción cerrada	Calendario en borrador, listo para publicar
Copa UAB 2026	Futsal, liga	Inscripción abierta	3 inscripciones aprobadas y 1 pendiente

Torneo	Disciplina y formato	Estado	Qué permite ver
Copa Baloncesto 2027	Baloncesto, 4 cupos	Inscripción abierta	Inscripciones pendientes y lista de espera
Torneo Relámpago de Tenis de Mesa	Otra, eliminación	Cancelado	Estado cancelado en el catálogo

8.3 Pruebas automáticas

```
npm test --prefix server     # 69 pruebas, no requieren MongoDB
cd client && npm run lint && npm run build
```

Tabla 17. *Pruebas automáticas por archivo*

Archivo	Qué verifica
`models.test.js`	Validadores e índices de los 5 modelos, bcrypt, OTP hasheado y su vencimiento
`middlewares.test.js`	`protect` (sin token, inválido, expirado, cuenta inactiva), `authorize` (RBAC) y manejo de errores
`api.test.js`	Endpoints con Supertest: salud, Swagger, registro, login (401, 403, 200 y expiración de 30 min), `/me`, recuperación de contraseña, RBAC (401/403), IDs inválidos y exportación PDF
`crypto.test.js`	Cifrado AES-256-GCM: ida y vuelta, determinismo, detección de alteraciones, datos antiguos y modelo `User`
`standings.test.js`	Reglas de puntuación por disciplina, tablas de voleibol y baloncesto, estado y avance del torneo en `GET /matches/standings`
`acta.test.js`	Marcador automático por suma de goles o puntos, rechazo de marcadores inconsistentes, empates, sets válidos y finalización automática del torneo
`seed.test.js`	Datos de demostración: validadores de Mongoose, índices únicos, plantillas válidas, árbitros sin solapamientos y estados de cada torneo
`services.test.js`	Envío de correos simulados y generación de PDF

Resumen de la ejecución de `npm test --prefix server` sobre la rama principal:

```
ℹ tests 69
ℹ suites 0
ℹ pass 69
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
```

8.4 Pruebas manuales realizadas (sobre el seed)

Tabla 18. *Pruebas manuales realizadas sobre los datos de demostración*

Área	Resultado
Autenticación	Login rechazado sin verificar (403), verificación OTP (200), `/me` con y sin token
Usuarios	Filtros, cambio de rol, desactivación y reactivación; no se puede cambiar el propio rol
Equipos	Alta con promoción a capitán, jugador duplicado (409), baja lógica
Inscripción	Aprobar y rechazar; equipo ya inscrito (409)
Calendario	Generación de 6 partidos, publicación, borrador oculto al público
Choques de horario	Árbitro solapado (409), reprogramación a fecha pasada (400)
Acta	Marcador que se suma solo con los goles o puntos por jugador, empate en baloncesto rechazado (400), sets inválidos en voleibol (400), acta cerrada (200), cerrada no editable (409), walkover 3-0, borrador recuperable

Área	Resultado
Posiciones	Tabla y goleadores actualizados al cerrar actas; columnas y puntos según la disciplina; campeón en torneos finalizados; llaves en eliminación directa; estados no iniciado, en curso y finalizado
Control de acceso	Pantallas por rol («Acceso restringido») y 403 en el servidor

8.5 Flujo de trabajo Git

Cada cambio se desarrolló en una rama `feature/*` creada desde `develop` y se integró con `git merge --ff-only`, de modo que el historial es lineal, sin commits de fusión, y conserva al autor de cada commit. `main` solo recibe lo que ya está en `develop`. El historial completo se ve en GitHub (Insights, Contributors).

Tabla 19. *Commits por integrante en el historial del repositorio*

Integrante	Commits	Áreas con más cambios
Samuel Eli Ramírez Mamani	15	Estructura del monorepo, informe y capturas, rutas y panel de administración
José Alcozer Martínez	10	Autenticación, torneos, pruebas automáticas, reglas por disciplina
Roger Huarachi Rojas	10	Pantallas del cliente, usuarios y equipos, datos de demostración
Kevin Abdias Flores Escobar	10	Controladores y rutas del servidor, servicios, panel del organizador

9. Conclusiones y trabajo futuro

9.1 Conclusiones técnicas

La separación en SPA, API REST y base documental permitió que cuatro personas trabajaran en paralelo: cada módulo del servidor (modelos, controladores y rutas) tiene su pantalla en el cliente y su sección en Swagger, lo que facilitó repartir el trabajo y revisar los cambios por área.

Las reglas que no pueden fallar viven en el servidor: elegibilidad de jugadores, cupos de inscripción, choques de horario, coherencia del acta y puntuación por disciplina. El cliente solo las anticipa para mostrar mensajes claros; quien use la API directamente recibe las mismas respuestas.

Los datos derivados no se almacenan. La tabla de posiciones, los goleadores y el marcador del acta se calculan a partir de los partidos y de las estadísticas por jugador, de modo que reabrir o corregir un acta no deja cifras desactualizadas.

El sistema cumple los 15 requisitos funcionales y los 8 no funcionales de la primera entrega. Lo que depende del entorno de producción (HTTPS, disponibilidad y respaldos) está justificado en la sección 3.3.

Tabla 20. *Evidencia por criterio de evaluación*

Criterio (peso)	Evidencia en el proyecto	Dónde verlo
Repositorio y estructura (15 %)	Monorepo `client/`, `server/`, `docs/`; `.gitignore` sin credenciales; tres `.env.example`; README con instrucciones; historial lineal con 4 autores	GitHub y sección 8.5
Autenticación JWT y RBAC (20 %)	JWT de 30 min con `id` y `rol`; bcrypt con salt >= 10; middlewares `protect` y `authorize`; 5 roles; cifrado AES-256-GCM	`server/middlewares/auth.js`, secciones 3.3 y 4.2
Persistencia en MongoDB (20 %)	5 colecciones con validadores, referencias, índices y datos de demostración reproducibles	Sección 5, `server/models/`, `server/seeds/`
Arquitectura y diagramas (15 %)	C4 nivel 2, secuencia de autenticación y modelo entidad-relación	Sección 4 y `docs/diagrams/`
API REST y Swagger (15 %)	45 endpoints con esquemas, parámetros, códigos de estado y roles	`/api-docs` y sección 6
Informe técnico (15 %)	Las 9 secciones solicitadas, matriz RF → endpoint y justificación de los RNF	Este documento

9.2 Lecciones aprendidas

Probar con cada rol y con datos realistas encontró más fallos que las pruebas unitarias: la pestaña de goleadores vacía en voleibol o el calendario abriendo en el mes más antiguo solo se notaron al cargar un seed con 9 torneos. Clonar el repositorio desde cero, siguiendo solo el README, fue la prueba más reveladora de todas.

Tabla 21. *Decisiones técnicas y alternativas descartadas*

Decisión	Alternativa descartada	Motivo
JWT sin estado en `Authorization: Bearer`	Sesiones en el servidor	API sin estado, simple de escalar y de documentar en Swagger
Posiciones y goleadores calculados al consultar	Tablas almacenadas y actualizadas	Evita datos derivados desincronizados al corregir actas
Marcador derivado de goles o puntos por jugador	Marcador manual con validación posterior	Elimina la posibilidad de actas descuadradas
Reglas de puntuación en un módulo (`services/reglas.js`)	Lógica fija de 3 puntos por victoria	Baloncesto y voleibol puntúan distinto; tabla, PDF y API comparten reglas

Decisión	Alternativa descartada	Motivo
AES-256-GCM determinista para el código estudiantil	Cifrado con IV aleatorio	Conserva el índice único y la búsqueda exacta; GCM detecta alteraciones
Correo SMTP configurable con modo simulado	Exigir un servidor SMTP	El proyecto se ejecuta sin credenciales; con SMTP envía correos reales
Seed determinista generado por una función pura	Datos aleatorios en cada ejecución	Resultados reproducibles y verificables con pruebas sin base de datos
Monorepo con integración `--ff-only`	Commits de fusión	Historial lineal que conserva a cada autor
Actualización por consulta cada 30 s	WebSockets	Menor complejidad y suficiente para el caso de uso

Tabla 22. *Problemas encontrados y cómo se resolvieron*

Problema detectado	Causa	Solución
Un árbitro podía quedar en dos partidos casi simultáneos	La validación comparaba la hora exacta de inicio	Choques detectados por intervalos de 90 minutos, respuesta 409
La API decía «escuchando» con el puerto ocupado	Express 5 ejecuta el callback de `listen` también ante errores	Se maneja el error de `listen` y se informa el puerto en uso
`npm run dev` fallaba en un clon limpio	`install:all` no instalaba las dependencias de la raíz	`install:all` instala raíz, servidor y cliente; verificado clonando `main`
Pestaña de goleadores vacía en voleibol	El diseño asumía goles en todas las disciplinas	Reglas y estadísticas por disciplina; la pestaña se oculta donde no aplica
Marcador distinto de la suma de goles por jugador	El acta pedía ambos datos por separado	Marcador calculado como suma; el servidor rechaza valores inconsistentes
Torneos de eliminación directa sin llegar a «finalizado»	Solo las ligas cerraban el torneo	El torneo finaliza al cerrarse la final
Texto del PDF partido en una columna angosta	PDFKit conserva la coordenada x de la última celda	Se restablece el margen izquierdo al terminar la tabla
Calendario abierto en el mes del partido más antiguo	Tomaba el primer partido de la lista	Abre en el mes del próximo partido, o del último si ya se jugaron todos
El organizador no podía listar árbitros	`GET /users` era exclusivo del administrador	Endpoint específico `GET /api/users/referees` para el organizador

9.3 Trabajo futuro

Tabla 23. *Mejoras pendientes*

Mejora	Situación actual
Generar las rondas siguientes de eliminación directa según los ganadores	El generador de calendario crea solo la primera ronda
Duración de partido por disciplina	Fija en 90 minutos
Autogoles y eventos minuto a minuto	Todo gol o punto se asigna a un jugador del equipo que anota
Actualización en tiempo real con WebSockets	La pantalla de posiciones consulta cada 30 s mientras el torneo está en curso
Pruebas de integración con base de datos en memoria y pruebas end-to-end	Pruebas con consultas simuladas; verificación manual de la interfaz
Despliegue con HTTPS, monitoreo, respaldos y rotación de claves de cifrado	Ejecución local o detrás de un proxy con HTTPS
Recordatorios de partido y cola de correos con reintentos	Correos puntuales sin reintentos